

Informe técnico de avances

CAPSTONE – INGENIERÍA EN INFORMATICA

EDUARDO BRAVO DIAZ

Contenido

Abstract.....	3
Español.....	3
English.....	3
Introducción	4
Desarrollo de ingeniería	5
Descripción del proyecto.....	5
Problemática y justificación	5
Objetivos del proyecto	5
Objetivo general	5
Objetivos específicos	5
Metodología de trabajo.....	6
Requerimientos del sistema	6
Requerimientos funcionales	7
Requerimientos no funcionales.....	7
Diseño de la solución tecnológica.....	7
Frontend.....	8
API Gateway.....	8
Backend y microservicios	8
Base de datos	8
Contenedor	8
Inteligencia artificial	8
Diseño de la base de datos.....	9
Avance del proyecto.....	10
Ajustes realizados a la propuesta	11
Dificultades identificadas.....	11
Facilitadores	11
Ajustes y acciones correctivas.....	12
Indicadores de calidad	13

Conclusions	14
Individual Reflection.....	15

Abstract

Español

El presente proyecto consiste en el desarrollo de un sistema web de Mesa de Ayuda, denominado Sistema MDA, cuyo propósito es centralizar y mejorar la gestión de solicitudes de soporte tecnológico. La solución busca facilitar la creación, asignación, seguimiento y resolución de tickets, permitiendo mantener un registro organizado de las solicitudes y de las interacciones entre usuarios y personal de soporte.

Para su desarrollo se considera una arquitectura basada en microservicios, utilizando React para la interfaz de usuario, Flask y Python para los servicios del backend, SQL Server para la persistencia de los datos y Docker para la configuración y ejecución de los componentes. Además, se contempla un API Gateway como punto central de comunicación entre el frontend y los servicios internos. Como proyección, se considera incorporar un componente de inteligencia artificial basado en aprendizaje automático para apoyar la clasificación de solicitudes.

Durante la etapa inicial se ha trabajado en la definición de la solución, el diseño de su arquitectura, la planificación de los componentes y la preparación del entorno de desarrollo. Entre los principales desafíos identificados se encuentran la configuración de los servicios, la conexión con la base de datos y la integración de los componentes independientes. Estos aspectos requieren pruebas y ajustes para asegurar una solución funcional, mantenible y escalable.

El proyecto busca aplicar los conocimientos adquiridos durante la carrera mediante el desarrollo de una solución de software que responda a una necesidad concreta de gestión de soporte tecnológico.

English

This project involves the development of a web-based Help Desk system called MDA, designed to centralize and improve the management of technical support requests. The solution aims to facilitate ticket creation, assignment, tracking, and resolution while maintaining an organized record of requests and interactions between users and support personnel.

The proposed solution uses a microservices-based architecture, with React for the user interface, Flask and Python for backend services, SQL Server for data persistence, and Docker for configuring and running the system components. An API Gateway is also planned to centralize communication between the frontend and internal services. As a future enhancement, the project considers integrating a machine learning component to support ticket classification.

During the initial stage, work has focused on defining the solution, designing the architecture, planning the system components, and preparing the development environment. The main challenges identified include service configuration, database connectivity, and integration between independent components. These aspects require additional testing and adjustments to ensure that the solution is functional, maintainable, and scalable.

The project aims to apply the knowledge acquired throughout the degree program by developing a software solution that addresses a practical need in technical support management.

Introducción

La gestión de solicitudes de soporte tecnológico es una actividad importante para las organizaciones, debido a que permite atender incidentes, resolver problemas y mantener la continuidad de los servicios informáticos. Cuando las solicitudes se gestionan mediante canales dispersos o procedimientos poco estandarizados, puede resultar difícil realizar su seguimiento, conocer su estado y mantener un historial de las acciones efectuadas.

En este contexto, el proyecto Sistema MDA propone desarrollar una plataforma web que permita administrar las solicitudes de soporte mediante tickets, centralizando la información y facilitando la comunicación entre los usuarios y el personal responsable de atenderlas.

Este informe presenta el avance del proyecto durante su etapa de desarrollo, describiendo la propuesta técnica, la metodología de trabajo, las decisiones de arquitectura, el diseño de datos, las dificultades identificadas y los ajustes considerados para continuar con la implementación.

Desarrollo de ingeniería

Descripción del proyecto

El Sistema MDA corresponde a una plataforma web orientada a la gestión de solicitudes de soporte. Su objetivo es disponer de un entorno centralizado donde los usuarios puedan registrar problemas, consultar el estado de sus solicitudes y mantener una comunicación relacionada con cada ticket.

La solución considera diferentes responsabilidades dentro del sistema, como la administración de usuarios, la autenticación, la gestión de tickets, el registro de mensajes y la asignación de solicitudes al personal correspondiente.

Adicionalmente, se proyecta incorporar capacidades de inteligencia artificial que permitan analizar los mensajes ingresados y sugerir una clasificación inicial de los tickets. Esta funcionalidad se considera una mejora progresiva y no se presenta como terminada en esta etapa.

Problemática y justificación

La falta de centralización de las solicitudes de soporte puede dificultar la trazabilidad de los incidentes, la priorización del trabajo y la identificación de las responsabilidades asociadas a cada solicitud.

El Sistema MDA busca abordar esta problemática mediante una plataforma que permita mantener los datos de los tickets en una base estructurada, registrar sus cambios de estado y asociar las comunicaciones con la solicitud correspondiente.

La implementación de esta solución se justifica por la necesidad de disponer de un mecanismo organizado para gestionar el ciclo de vida de los tickets, desde su creación hasta su resolución y cierre. Asimismo, una arquitectura modular permite separar responsabilidades técnicas y facilitar el mantenimiento futuro de la aplicación.

Objetivos del proyecto

Objetivo general

Desarrollar una plataforma web de Mesa de Ayuda que permita centralizar y administrar solicitudes de soporte tecnológico mediante la gestión de usuarios, tickets, estados y comunicaciones, utilizando una arquitectura modular que facilite su mantenimiento y evolución.

Objetivos específicos

- Diseñar una arquitectura de software que permita separar las principales responsabilidades funcionales del sistema.

- Definir una estructura de datos que permita almacenar y relacionar usuarios, tickets, estados, prioridades y mensajes.
- Desarrollar los componentes de frontend y backend necesarios para implementar las funcionalidades principales.
- Configurar el entorno de ejecución y facilitar la integración de los servicios mediante herramientas de contenerización.
- Planificar pruebas funcionales y de integración para verificar el comportamiento de la solución.
- Evaluar la incorporación futura de un modelo de aprendizaje automático que apoye la clasificación de los tickets.

Metodología de trabajo

Para organizar el desarrollo se utiliza un enfoque basado en Scrum, adaptado a las características de un proyecto realizado individualmente. El trabajo se divide en etapas y actividades que permiten avanzar de manera progresiva, revisar los resultados obtenidos y ajustar la planificación cuando se presentan dificultades técnicas.

La planificación general considera tres fases:

Fase	Periodo planificado	Actividades principales
Fase 1	Semanas 1 a 4	Levantamiento de requerimientos, análisis y diseño de la solución.
Fase 2	Semanas 5 a 15	Desarrollo de funcionalidades, configuración e integración de componentes.
Fase 3	Semanas 16 a 18	Pruebas, correcciones, documentación y cierre del proyecto.

La planificación se utiliza como referencia para organizar las tareas y revisar su cumplimiento. Debido a que el desarrollo puede presentar dificultades técnicas, las actividades se pueden reordenar o extender cuando sea necesario, procurando mantener los objetivos principales.

Al trabajar individualmente, la organización de tareas, la implementación, las pruebas y la documentación son responsabilidad del estudiante. Por ello, resulta importante mantener un registro de avances, problemas encontrados y decisiones técnicas adoptadas.

Requerimientos del sistema

Los requerimientos iniciales se organizan según las principales funcionalidades esperadas de la plataforma.

Requerimientos funcionales

RF01. Gestión de usuarios: permitir el registro y la administración de usuarios del sistema, considerando sus datos de identificación y rol.

RF02. Autenticación: permitir el inicio y cierre de sesión, controlando el acceso a las funcionalidades según la identidad y los permisos del usuario.

RF03. Creación de tickets: permitir registrar solicitudes de soporte con información descriptiva del problema.

RF04. Gestión de tickets: permitir consultar las solicitudes y administrar su estado, prioridad, tipo y asignación, según los permisos correspondientes.

RF05. Mensajería: permitir registrar mensajes asociados a un ticket para mantener la comunicación y la trazabilidad de las acciones.

RF06. Administración de estados: contemplar estados que representen el ciclo de vida de una solicitud, como Abierto, En proceso, Pendiente, Resuelto, Cerrado, Reabierto y Cancelado.

RF07. Clasificación asistida: considerar, como funcionalidad futura, el uso de un modelo de aprendizaje automático para sugerir atributos de clasificación a partir del mensaje inicial del usuario.

Requerimientos no funcionales

- Seguridad: controlar la autenticación y autorización de las operaciones.
- Mantenibilidad: organizar el código en componentes con responsabilidades diferenciadas.
- Escalabilidad: permitir que los servicios puedan evolucionar de forma independiente cuando las necesidades del sistema lo requieran.
- Integridad de datos: mantener relaciones consistentes entre usuarios, tickets, mensajes y catálogos.
- Desplegabilidad: utilizar herramientas que faciliten la configuración y ejecución del sistema en distintos entornos.
- Trazabilidad: conservar información relevante sobre las solicitudes y sus cambios.

Estos requerimientos constituyen una base de trabajo que deberá validarse y ajustarse a medida que avance el desarrollo.

Diseño de la solución tecnológica

La arquitectura propuesta utiliza una separación entre la interfaz de usuario, el punto de entrada de las solicitudes y los servicios de negocio.

Frontend

Se utiliza React para construir la interfaz web con la que interactúan los usuarios. Este componente se orienta a presentar la información, capturar las solicitudes y mostrar los resultados devueltos por el backend.

API Gateway

El API Gateway actúa como punto central de entrada para las solicitudes del frontend. Su responsabilidad es dirigir las peticiones hacia el servicio correspondiente, evitando que la interfaz tenga que gestionar directamente la ubicación interna de cada microservicio.

Backend y microservicios

Los servicios del backend se desarrollan utilizando Python y Flask. La separación por responsabilidades contempla componentes para autenticación, gestión de tickets, mensajes y notificaciones. Esta organización busca reducir el acoplamiento entre las funcionalidades y facilitar su mantenimiento.

La división en microservicios implica que cada componente debe configurarse y comunicarse correctamente con los demás. Por esta razón, la integración constituye una actividad técnica importante dentro del proyecto.

Base de datos

SQL Server se considera el motor relacional para almacenar la información estructurada de la aplicación, incluyendo usuarios, roles, tickets y catálogos relacionados. Las relaciones entre las entidades permiten mantener la integridad de la información y facilitar las consultas necesarias para las funcionalidades del sistema.

Contenedor

Docker se utiliza como herramienta para preparar el entorno de ejecución de los componentes. Mediante contenedores y archivos de configuración se busca estandarizar la instalación y reducir las diferencias entre el entorno de desarrollo y el de despliegue.

Inteligencia artificial

Como evolución del proyecto, se contempla desarrollar un modelo de aprendizaje automático que permita analizar los mensajes iniciales de los usuarios y sugerir atributos como la subcategoría, el tipo de ticket y la prioridad.

Para ello, será necesario preparar un conjunto de datos etiquetados, seleccionar un algoritmo adecuado y evaluar sus resultados antes de integrar las predicciones

al flujo de creación de tickets. Esta funcionalidad debe considerarse una línea de desarrollo posterior mientras no se encuentre implementada y validada.

Diseño de la base de datos

El modelo de datos se orienta a mantener separada la información de los usuarios, las solicitudes y los catálogos utilizados para clasificar los tickets.

Entre las entidades principales consideradas se encuentran:

Entidad	Propósito
Usuario	Almacenar los datos de identificación y acceso de los usuarios.
Rol	Definir los perfiles y responsabilidades dentro del sistema.
Ticket	Registrar la solicitud, su título, estado, prioridad, tipo y usuarios relacionados.
Mensaje	Almacenar las comunicaciones asociadas a una solicitud.
Estado	Mantener el catálogo de estados posibles de los tickets.
Prioridad	Definir los niveles de prioridad utilizados en la gestión.
Tipo de ticket	Clasificar las solicitudes según su naturaleza.
Subcategoría	Permitir una clasificación más específica de los incidentes.

Las entidades deben relacionarse mediante claves primarias y foráneas, evitando duplicar innecesariamente la información de los catálogos. La estructura definitiva debe validarse junto con las funcionalidades que se implementen.

Este diseño busca favorecer la consistencia de los datos, facilitar las consultas y permitir que el sistema mantenga un historial organizado de las solicitudes.

Avance del proyecto

Durante la etapa inicial, el trabajo se ha orientado a definir la solución, establecer sus componentes principales y preparar las bases técnicas necesarias para continuar con el desarrollo.

Los avances que deben documentarse en esta etapa incluyen:

- Definición del propósito y alcance del Sistema MDA.
- Identificación de los principales requerimientos funcionales y no funcionales.
- Selección de tecnologías para frontend, backend, persistencia y ejecución.
- Diseño de una arquitectura basada en servicios independientes y un API Gateway.
- Definición inicial de las entidades necesarias para gestionar usuarios y tickets.
- Preparación del entorno de desarrollo y configuración de los componentes que se utilizarán en la implementación.

El progreso de cada actividad debe respaldarse mediante evidencia verificable, como diagramas, archivos de configuración, scripts SQL, capturas de la aplicación o pruebas realizadas.

Las funcionalidades que aún presenten problemas de configuración, conexión o integración deben mantenerse identificadas como pendientes de validación. De esta manera, el informe refleja el estado real del proyecto y permite establecer acciones concretas para continuar avanzando.

Ajustes realizados a la propuesta

Durante el desarrollo de una solución de software pueden surgir dificultades que obliguen a revisar las decisiones iniciales. En el caso del Sistema MDA, la arquitectura basada en microservicios requiere configurar distintos componentes y asegurar que puedan comunicarse correctamente.

Dificultades identificadas

Una de las principales dificultades técnicas corresponde a la configuración del entorno de ejecución y la integración entre los servicios del backend. La separación de componentes exige verificar las dependencias, las rutas de importación, la configuración de los contenedores y la comunicación entre servicios.

También es importante validar la conexión con SQL Server, ya que los parámetros de conexión, la disponibilidad del servidor y la configuración de red pueden afectar el funcionamiento de las operaciones que dependen de la base de datos.

Estas dificultades evidencian la necesidad de realizar pruebas por componente antes de evaluar el funcionamiento integral de la plataforma.

Facilitadores

Entre los elementos que facilitan el desarrollo se encuentran la utilización de tecnologías conocidas y ampliamente utilizadas, la separación de responsabilidades y la posibilidad de probar los servicios de manera independiente.

El uso de herramientas como Docker permite organizar la ejecución de los componentes, mientras que una estructura modular facilita localizar errores y realizar cambios sin modificar necesariamente toda la aplicación.

La documentación técnica y el registro de los problemas encontrados también contribuyen a mantener un proceso de trabajo ordenado.

Ajustes y acciones correctivas

A partir de las dificultades identificadas, se consideran los siguientes ajustes:

- Revisar la estructura de carpetas e importaciones de cada microservicio para asegurar que los módulos puedan cargarse correctamente.
- Verificar las variables de entorno y los parámetros de conexión utilizados por los servicios.
- Comprobar la conectividad con SQL Server antes de probar funcionalidades que dependan de consultas o escrituras en la base de datos.
- Construir y ejecutar los contenedores de forma individual antes de realizar pruebas de integración.
- Validar progresivamente las rutas del API Gateway y las respuestas de cada servicio.
- Actualizar la planificación cuando una actividad requiera más tiempo del previsto.

Estos ajustes permiten abordar los problemas de manera sistemática y reducen el riesgo de intentar validar el sistema completo sin haber comprobado previamente sus componentes.

En relación con la retroalimentación docente, las observaciones recibidas deben incorporarse en esta sección cuando correspondan. Si todavía no se ha recibido retroalimentación formal, se debe indicar esa situación sin atribuir recomendaciones a terceros.

Indicadores de calidad

Para evaluar la calidad de la solución se proponen indicadores que permitan verificar tanto el avance técnico como el comportamiento del sistema.

Indicador	Forma de verificación	Criterio esperado
Cumplimiento de requerimientos	Comparar las funcionalidades desarrolladas con los requerimientos definidos.	Cada requerimiento debe tener un estado y una evidencia asociada.
Funcionamiento de los servicios	Ejecutar pruebas individuales sobre los endpoints disponibles.	Los servicios deben responder de acuerdo con el comportamiento esperado.
Conectividad con la base de datos	Probar operaciones de lectura y escritura autorizadas.	Las operaciones deben ejecutarse sin errores de conexión y respetar la integridad de los datos.
Integración entre componentes	Probar solicitudes que atraviesen el API Gateway y lleguen al servicio correspondiente.	La petición debe alcanzar el destino correcto y devolver una respuesta consistente.
Control de acceso	Probar rutas protegidas con sesiones válidas e inválidas.	El sistema debe permitir o rechazar las solicitudes según las reglas de autenticación y autorización.
Mantenibilidad	Revisar la separación de responsabilidades y la organización del código.	Los componentes deben tener responsabilidades identificables y una estructura comprensible.
Gestión de incidencias técnicas	Mantener un registro de errores, causas y correcciones.	Los problemas detectados deben contar con seguimiento y acciones definidas.

Estos indicadores corresponden a criterios de evaluación propuestos. Su cumplimiento debe determinarse mediante pruebas y evidencias; no se considera acreditado únicamente por haber definido el indicador.

Conclusions

The MDA project establishes the foundation for a web-based Help Desk platform focused on improving the organization and traceability of technical support requests. The proposed architecture separates the user interface, API Gateway, backend services, and data persistence, providing a modular structure that can support future improvements.

The initial development stage has highlighted the importance of validating the technical environment before attempting to integrate all system components. In particular, service configuration, database connectivity, and communication between microservices are essential aspects that require systematic testing.

The next steps should prioritize stabilizing the development environment, verifying each service independently, and gradually testing the complete request flow. It is also necessary to maintain evidence of progress and update the project plan when technical difficulties affect the estimated duration of tasks.

Overall, the project provides an opportunity to apply software engineering principles to a practical problem. Its success will depend not only on implementing the planned features, but also on verifying their operation, maintaining data consistency, and documenting the decisions and adjustments made throughout development.

Individual Reflection

Developing the MDA project individually has required me to organize the different activities involved in a software engineering project, including requirements analysis, architecture design, implementation, configuration, and documentation. This process has helped me understand the importance of planning and defining clear responsibilities for each component.

One of the main challenges has been preparing the technical environment and ensuring that the backend services and database can work together correctly. These difficulties have shown me that writing the application code is only one part of the development process. Configuration, testing, troubleshooting, and integration are also essential to delivering a functional solution.

To address these challenges, I need to continue validating the components independently, documenting errors, and applying corrections progressively. I also need to review the project schedule regularly so that unexpected technical problems can be incorporated into the planning.

This experience has strengthened my understanding of modular architecture, database design, and the importance of testing. Moving forward, my priority is to consolidate the core functionality of the Help Desk system, gather evidence of its operation, and ensure that future improvements, including machine learning, are introduced only after the main components have been validated.